

Как дерево записывают в условии задачи.

Справочник по нотации, а не разбор задачи. Запись одинакова во всех задачах про деревья, поэтому она вынесена отдельно.

Запись по уровням.

Дерево задают массивом: обход **в ширину слева направо**, пропущенные потомки записаны как `null`.

Вход: `root = [3, 9, 20, null, null, 15, 7]`.



Читается это так: первый элемент — корень, дальше идут потомки узлов в том порядке, в каком сами узлы встретились.

Как нарисовать дерево по записи.

Правило простое: **ячейки идут парами, и одна пара — это два потомка одного узла**; в каждой ячейке либо значение потомка, либо `null`. Узлы получают свою пару по порядку появления, сверху вниз и слева направо. Пустой узел (`null`) пары не получает: детей у него быть не может.

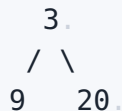
Дальше нужен только список тех, **кому ещё не выдали детей**. Разберём `[3, 9, 20, null, null, 15, 7]` по шагам.

Начало: первое число — корень.

3.

Кому ещё не выдали детей: 3.

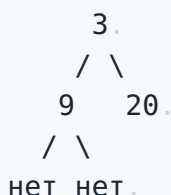
Шаг 1: берём тройку, выдаём ей 9 и 20.



Тройка своё получила — вычёркиваем её из списка. Появились двое новых, дописываем их в конец.

Кому ещё не выдали детей: 9, 20.

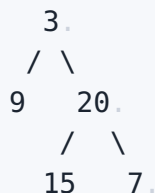
Шаг 2: берём девятку, выдаём ей null и null.



Девятку вычёркиваем. Новых узлов не появилось: оба потомка пустые.

Кому ещё не выдали детей: 20.

Шаг 3: берём двадцатку, выдаём ей 15 и 7.



Двадцатку вычёркиваем, дописываем в конец 15 и 7.

Кому ещё не выдали детей: 15, 7.

Конец: массив кончился.

Выдавать больше нечего, значит 15 и 7 детей не получают — они листья. Дерево готово.

Почему шагов всего три.

Не потому, что узлов три, а потому, что закончился массив. Узлы 15 и 7 так и остались в списке ожидающих: до них раздача не дошла.

И отсюда же ответ на вопрос, почему список на каждом шаге разный. Это не перечень всех узлов дерева, а список тех, кому ещё не выдали детей: на каждом шаге одного вычёркивают, а новых дописывают в конец.

В коде этот список — обычная очередь (`Queue`): берут с начала, дописывают в конец. Порядок «берём с начала, дописываем в конец» и даёт раздачу уровень за уровнем — это и есть обход в ширину.

Два правила, из которых всё следует.

1. **непустому** узлу отводятся две ячейки — под левого потомка и под правого, даже если в обеих `null`. Раздача идёт по очереди и обрывается там, где кончилась запись: последнему узлу может достаться одна ячейка, а тем, кто стоит в очереди за ним, — ни одной. Недостающая ячейка значит то же, что `null`: потомка нет;
2. `null` в список ожидающих не попадает, поэтому его собственных потомков в массиве нет вовсе.

Второе правило и делает запись компактной: пустые ветки не разрастаются.

Хвостовые `null` принято опускать. Записи `[1, 2]` и `[1, 2, null]` описывают одно и то же дерево.

Пример с пропусками внутри.

`[1, null, 2, null, 3]` — тот же порядок действий.

Начало. Первое число — корень.

1.

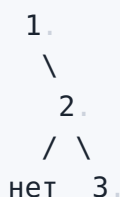
Кому ещё не выдали детей: 1.

Шаг 1. Берём единицу, выдаём ей `null` и 2: левого потомка нет, правый — двойка.

```
  1.
 /  \
нет  2.
```

Кому ещё не выдали детей: 2 — пустой потомок в список не идёт.

Шаг 2. Берём двойку, выдаём ей `null` и 3.



Кому ещё не выдали детей: 3.

Конец. Массив кончился, тройка остаётся листом:



Здесь важны два места. Первое: `null` занял ровно одну ячейку — место левого потомка единицы — и на этом закончился, дальше чтение продолжилось как обычно. Второе: в список ожидающих `null` не попал, поэтому на следующем шаге брали двойку, а не его.

Откуда известно, что потомков не больше двух.

Не из массива, а из **типа входа**. На вход подаётся `TreeNode`, а у него всего два поля-ссылки:

```
public class TreeNode {
    public int val;
    public TreeNode left;
    public TreeNode right;
}
```

Третьего потомка негде хранить. Название задачи говорит о том же: **binary tree** — двоичное дерево, у узла не больше двух детей. Отсюда и пара ячеек на узел в записи: пара — потому что полей ровно два.

Из этого следует и граница применимости: запись по уровням годится **только для двоичного дерева**. Другие структуры в условиях задают иначе, и видно это по сигнатуре:

Что за структура	Как задают в условии
двоичное дерево	массив по уровням с <code>null</code> — то, что разбирается здесь
дерево с любым числом детей	список детей у каждого узла, часто с <code>null</code> как разделителем групп
произвольный граф	список рёбер <code>[[0,1],[1,2]]</code> или список смежности
сетка, лабиринт, острова	двумерный массив, соседство по четырём направлениям

Два способа положить дерево в массив

Массивом дерево можно записать **двумя разными способами**, и оба законны. Путаница возникает потому, что на некоторых деревьях они дают одинаковый результат.

Способ 1: полное представление, по формуле

Ячейка отводится каждой позиции дерева, включая пустые. Тогда работает арифметика:

потомки узла с индексом i → соответствует $2i + 1$ и $2i + 2$
 родитель узла с индексом i → соответствует $(i - \text{минус } 1) / 2$

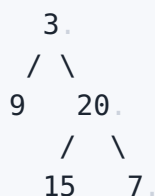
Так устроена двоичная куча — `PriorityQueue` в Java хранит элементы именно так.

Способ 2: запись по уровням, очередью

Пишутся только реально существующие узлы и их прямые потомки. Потомков у `null` не бывает, поэтому и в массиве их нет. Так задают деревья в условиях задач, и разбирается такая запись очередью — как в разделах выше.

Когда способы совпадают

Возьмём дерево из первого примера — оно не законченное, у девятки потомков нет:



По формуле: корень 3 на индексе 0; его дети на 1 и 2 — 9 и 20; дети девятки на 3 и 4 — оба пустые; дети двадцатки на 5 и 6 — 15 и 7.

[3, 9, 20, null, null, 15, 7].

Это в точности запись из условия. **Совпало** — потому что пропуски здесь только на последнем уровне и никого не сдвигают. Именно из-за таких случаев кажется, будто формула работает всегда.

На дереве с пропуском внутри — расходятся.



Способ	Запись	Ячеек
запись по уровням	[1, null, 2, null, 3]	5
полное представление	[1, null, 2, null, null, null, 3]	7

В полном представлении у null на индексе 1 всё равно резервируются места под детей — индексы 3 и 4, — хотя детей у него быть не может. Из-за этого тройка съезжает с индекса 4 на индекс 6.

Отсюда правило: **применять формулу к записи из условия нельзя**. В ней у пустых узлов места под детей не зарезервированы, и индексы после первого же внутреннего пропуска перестают сходиться.

Почему в условиях выбран второй способ.

Размер полного представления определяется не числом узлов, а высотой. Пусть m — сколько нужно ячеек, h — высота дерева, считая в узлах, как принято в `BinaryTree.md`. Тогда $m = 2^h - 1$.

Дерево из ста узлов, вытянутое в цепочку, имеет высоту сто — и потребует $2^{100} - \text{минус } 1$ ячеек вместо ста.

	Запись по уровням	Полное представление
что в массиве.	реально существующие узлы и их прямые потомки.	каждая позиция дерева, включая пустые.
как восстановить.	очередью.	формулой $2i + 1$, $2i + 2$.
размер зависит от.	числа узлов.	высоты дерева.
цепочка из 10^4 узлов.	около $2 \cdot 10^4$ ячеек.	$2^{10000} - \text{минус } 1$ ячеек — столько памяти не существует.
где встречается	условия задач, любая форма дерева.	<code>PriorityQueue</code> , двоичная куча — там дерево заведомо законченное

Практический вывод: увидев в условии массив с `null`, **не пересчитывать индексы, а восстанавливать дерево очередью.**

Обратная задача: записать дерево массивом.

То же самое, только наоборот. Список ожидающих ведётся так же — это узлы, чьих детей ещё не записали:

1. записать значение корня, поставить корень в список ожидающих;
2. взять первого из списка, дописать в массив **оба** его потомка — значение или `null`;
3. непустых потомков дописать в конец списка, самого узла вычеркнуть;
4. повторять, пока список не опустеет;
5. в конце отбросить хвостовые `null`.

Проверить себя можно по числу ячеек. До того как отброшены хвостовые `null`, у дерева из n узлов получается $2n + 1$ ячейка: значение корня плюс по две ячейки на каждый узел. После шага 5

остаются только те `null`, за которыми дальше в записи есть хотя бы одно значение.

Где это в коде проекта

`TreeNode.fromLevelOrder` в пакете `block06_trees_graphs` собирает дерево ровно по правилам выше. Нужен он только тестам: примеры из условия задаются в том же виде, в каком написаны на LeetCode, и не приходится строить дерево вручную через конструкторы.

Запись, в которой значения остались, а выдавать их некому (`[1, null, null, 5]`: после двух `null` список ожидающих пуст, а пятёрка ещё не прочитана), метод считает ошибкой и бросает `NoSuchElementException`. На LeetCode такие записи не встречаются.

`TreeNode.toLevelOrder` — обратная операция из раздела выше: дерево в запись, хвостовые `null` отброшены. Нужен тестам самого `TreeNode`: круг «из записи в дерево и обратно в запись» должен вернуть исходную запись.

Рядом лежит `TreeNode.chainOfLength` — он строит вырожденное дерево-цепочку, у которой есть только правые потомки, а значения идут от нуля по порядку. В записи по уровням такая цепочка выглядит как `[0, null, 1, null, 2, ...]`.

Тесты всех трёх методов — в `main()` класса `TreeNode`.

fromLevelOrder : ИЗ ЗАПИСИ В ДЕРЕВО.

```
public static TreeNode fromLevelOrder(Integer... values) {
    if (values.length == 0 || values[0] == null) {
        return null;
    }

    TreeNode root = new TreeNode(values[0]);
    Queue<TreeNode> nodesToVisit = new ArrayDeque<>();
    nodesToVisit.add(root);

    int position = 1;
    while (position < values.length) {
        TreeNode parent = nodesToVisit.remove();

        /*
         * Ячейка под левого потомка есть всегда – это гарантирует условие
         * цикла. Само значение может оказаться null, тогда потомка нет.
         */
        Integer leftValue = values[position];
        position++;
        if (leftValue != null) {
            parent.left = new TreeNode(leftValue);
            nodesToVisit.add(parent.left);
        }

        /* А ячейки под правого может не быть вовсе: массив мог кончиться
        на левом. */
        if (position < values.length) {
            Integer rightValue = values[position];
            position++;
            if (rightValue != null) {
                parent.right = new TreeNode(rightValue);
                nodesToVisit.add(parent.right);
            }
        }
    }
    return root;
}
```

Что происходит в коде, строка за строкой.

Метод читает запись слева направо и ни разу не возвращается назад: индекс `position` только растёт. Дерево при этом строится не рекурсией сверху вниз, а очередью `nodesToVisit` — той самой, которая в разделах выше называлась списком ожидающих.

`if (values.length == 0 || values[0] == null)` — два вида записи означают пустое дерево: пустая запись и запись `[null]`. Обе дают `null`. Порядок условий важен: у пустого массива ячейки с индексом 0 нет, и обращение `values[0]` бросило бы `ArrayIndexOutOfBoundsException`. Оператор `||` вычисляет второе условие только тогда, когда первое ложно, то есть когда ячейка точно есть.

`TreeNode root = new TreeNode(values[0])` — нулевая ячейка записи это корень. Он создаётся отдельно от цикла, потому что цикл создаёт потомков, а корень ничьим потомком не является.

`nodesToVisit.add(root)` — корень встаёт в очередь: ему ещё не выдали потомков.

`int position = 1` — индекс ячейки записи, которую предстоит прочитать. Нулевую уже прочитал корень, поэтому счёт начинается с единицы.

`while (position < values.length)` — цикл идёт не по узлам дерева и не по уровням, а по непрочитанным ячейкам записи. Он останавливается, когда запись кончилась, и в очереди при этом могут остаться узлы, которым потомков так и не выдали. Это не ошибка: такие узлы — листья.

`TreeNode parent = nodesToVisit.remove()` — первая строка итерации: снять с головы очереди узел, которому сейчас выдадут потомков. Взял `remove`, а не `poll`: если очередь пуста, запись требует выдать потомков несуществующему узлу, и метод останавливается с `NoSuchElementException`. С `poll` вместо этого в `parent` попал бы `null`, и метод либо упал бы дальше по коду с `NullPointerException`, либо молча дочитал бы запись до конца и вернул неправильно собранное дерево.

`Integer leftValue = values[position]` и `position++` — прочитана ячейка под левого потомка, индекс сдвинут на следующую. Тип `Integer`, а не `int`, потому что в ячейке может лежать `null`.

`if (leftValue != null) { parent.left = new TreeNode(leftValue); nodesToVisit.add(parent.left); }` — если значение есть, создаётся узел, привязывается к родителю слева и встаёт в конец очереди: ему тоже когда-нибудь выдадут потомков. Если в ячейке `null`, не делается

ничего: поле `parent.left` и так равно `null`, а пустой узел в очередь не идёт — это второе из двух правил записи, разобранных выше.

`if (position < values.length)` — а вот ячейки под правого потомка может не быть. Почему так — отдельным разделом ниже.

Внутри этой проверки повторяется то же самое: прочитать `values[position]`, сдвинуть `position`, и если значение не `null` — создать узел, привязать справа, поставить в конец очереди.

`return root` — цикл кончился, возвращается ссылка на корень. Всё дерево держится на ней.

Что делает каждая переменная.

Переменная.	Что хранит.	Зачем она нужна.
<code>values</code> .	запись дерева по уровням, вместе с <code>null</code> .	вход метода; тип <code>Integer</code> , а не <code>int</code> , ради <code>null</code> в ячейках.
<code>root</code> .	корень будущего дерева.	создаётся до цикла, потому что родителя у него нет; он же возвращается в конце.
<code>nodesToVisit</code> .	очередь узлов, которым потомков ещё не выдали.	список ожидающих из разделов выше; порядок FIFO и даёт раздачу сверху вниз, слева направо.
<code>position</code> .	индекс ячейки записи, которую предстоит прочитать.	единственный счётчик чтения; только растёт, к прочитанному метод не возвращается.
<code>parent</code> .	узел, снятый с головы очереди на текущей итерации.	ему выдаются оба потомка.
<code>leftValue</code> .	значение из ячейки под левого потомка, может быть <code>null</code> .	по нему решается, создавать ли левого потомка.
<code>rightValue</code> .	то же для правого потомка.	объявлена внутри <code>if</code> , потому что ячейки под правого потомка может не быть вовсе.

Имя `nodesToVisit` — «узлы, которые предстоит посетить» — одно на весь блок: так же названа очередь в `LevelOrderTraversal`, в `TreeTraversals` и в `MaxDepthBinaryTreeIterative`. Меняется только то, что значит «посетить»: здесь — выдать снятому узлу двух потомков из записи, в задаче 102 — записать его значение в ответ.

Назвать эту очередь «родителями» было бы неточно: узел, пока он в очереди, родителем ещё не стал и может им не стать вовсе. Листья 15 и 7 из разбора ниже так и остаются в очереди — запись кончилась раньше, чем до них дошла раздача. Локальная переменная `parent` внутри цикла, наоборот, названа верно: снятый узел получает потомков прямо на этой итерации.

Почему ячейка под левого потомка есть всегда, а под правого может не быть.

Всё держится на том, что именно проверяет условие цикла. `position < values.length` означает «есть хотя бы одна неп прочитанная ячейка». Раз итерация началась, такая ячейка есть — и по правилам записи это ячейка под левого потомка снятого узла. Поэтому обращение `values[position]` в начале итерации безопасно, и отдельная проверка перед ним не нужна: её уже сделал `while`.

После чтения левой ячейки `position` вырос на единицу, и гарантия кончилась: запись могла оборваться ровно здесь.

Оборваться она может потому, что хвостовые `null` принято опускать. Полная запись содержит после корня чётное число ячеек: значения выдаются по два на узел, левое и правое. Но запись `[1, 2, null]` сокращают до `[1, 2]`, и тогда после корня остаётся одна ячейка — половина такой двойки.

Запись.	Что выдаётся корню 1.	Ячеек в записи.	Ячеек после корня.
<code>[1, 2, null]</code> .	2 и <code>null</code> .	три.	две.
<code>[1, 2]</code> .	2 и ничего.	две.	одна.

Отсутствующая ячейка и записанный в ячейке `null` дают один результат: правого потомка нет. Разница только в том, что в первом случае это сказано явно, а во втором — умолчанием. Поэтому `if (position < values.length)` не обрабатывает особый случай, а доводит умолчание до того же исхода: ничего не создавать.

Отсюда же видно, что оборваться запись может только на ячейке под правого потомка, а не под левого. Если бы она оборвалась на левой, неп прочитанных ячеек не осталось бы и итерация просто не началась бы.

Заодно это отвечает, почему у левой ячейки проверка не нужна, а у правой нужна: условие цикла проверяет ровно одну ячейку вперёд, а читает метод за итерацию две.

Почему снятый узел — тот самый родитель.

В коде нигде не сказано, какому узлу принадлежит прочитанная пара ячеек: ни номера родителя, ни ссылки назад в запись нет. Соответствие держится на трёх правилах, и достаточно убрать любое, чтобы оно разошлось. Пусть `k` — номер итерации цикла.

1. Очередь работает по правилу FIFO: узлы снимаются в том порядке, в каком их клали. Значит `k`-й снятый узел — это `k`-й положенный.
2. Кладут в очередь ровно в том порядке, в каком читают запись: корень до цикла, дальше на каждой итерации сначала левый потомок, потом правый. Значит `k`-й положенный узел — это `k`-й непустой узел в порядке появления в записи.
3. `null` в очередь не идёт, и в записи пустому узлу пара ячеек не выделяется. Оба счёта пропускают одни и те же места.

Отсюда инвариант цикла: **перед `k`-й итерацией в голове очереди стоит `k`-й по порядку непустой узел, а `position` указывает на первую ячейку `k`-й пары.**

Перед первой итерацией это верно: в очереди лежит корень — первый узел, `position` равен единице — это первая пара. Итерация снимает `k`-й узел, читает `k`-ю пару и дописывает его непустых потомков в хвост, то есть ровно тех, кто идёт в записи следующим. Значит перед итерацией с номером `k + 1` в голове очереди стоит узел с номером `k + 1`,

а `position` — на паре с номером `k + 1`. Оба счёта двигаются на единицу за итерацию и разойтись не могут: это один и тот же счёт, названный двумя способами.

Поэтому «снятый узел — родитель для прочитанной ячейки» в коде не проверяется. Это следствие того, что два порядка совпадают.

Убрать третье правило — и совпадение ломается. Пусть `null` тоже кладут в очередь. Запись `[1, null, 2, null, 3]`, правильное дерево: у `1` левого потомка нет, справа `2`; у `2` справа `3`.

Шаг	Снятый узел	Что прочитано	Что вышло бы
итерация 1.	1.	ячейка 1 = <code>null</code> , ячейка 2 = <code>2</code> .	в очередь встали бы пустой узел и <code>2</code> .
итерация 2.	пустой узел	ячейка 3 = <code>null</code> , ячейка 4 = <code>3</code> .	тройка досталась бы пустому узлу, а <code>2</code> осталась бы без потомков.

Запись на этом кончается, и дерево получилось бы другим. В Java до конца дело не доходит: на второй итерации `parent` равен `null`, и строка `parent.right = new TreeNode(rightValue)` бросает `NullPointerException`. Подмену родителя ловит тест `skipInside` — он утверждает, что `skipInside.right.right.val` равно `3`.

В задаче 102 тот же вопрос разобран с другой стороны: там очередь тоже ничего не помнит о родителях, и раздел «Почему узлы одного уровня у разных родителей не перепутаются» объясняет, почему потомки разных родителей не смешиваются в один уровень.

Разбор по шагам.

Запись `[3, 9, 20, null, null, 15, 7]`, дерево из первого примера:

Шаг.	Снятый узел.	Что прочитано.	position в конце шага.	Очередь nodesToVisit в конце шага.
старт.	— нет.	— нет.	1.	3.
итерация 1.	3.	ячейка 1 = 9 , создан левый потомок; ячейка 2 = 20 , создан правый потомок.	3.	9 , 20 .
итерация 2.	9.	ячейка 3 = null , левого потомка нет; ячейка 4 = null , правого потомка нет.	5.	20 .
итерация 3.	20 .	ячейка 5 = 15 , создан левый потомок; ячейка 6 = 7 , создан правый потомок.	7.	15 , 7 .

После третьей итерации `position` равен семи, и в записи ровно семь ячеек. Условие цикла ложно, четвёртая итерация не начинается, метод возвращает корень.

Итераций три, а узлов в дереве пять. Цикл считает не узлы дерева, а прочитанные ячейки: раздачу получили три узла — 3, 9 и 20. Узлы 15 и 7 так и остались в очереди, потому что запись кончилась раньше, чем дошло до них. Поэтому очередь в конце работы не пуста, и проверять её на пустоту после цикла не нужно.

Это ровно то, что в разделе «Почему шагов всего три» сказано словами, только теперь видно, какая строка кода за что отвечает.

Те же шаги на двух других записях:

Запись	Шаг	Снятый узел	Что прочитано	Очередь nodesToVisit в конце шага
[1, 2]	итерация 1.	1.	ячейка 1 = 2, создан левый потомок; ячейки под правого потомка нет.	2.
[1, null, 2, null, 3]	итерация 1.	1.	ячейка 1 = null, левого потомка нет; ячейка 2 = 2, создан правый потомок.	2.
[1, null, 2, null, 3]	итерация 2.	2.	ячейка 3 = null, левого потомка нет; ячейка 4 = 3, создан правый потомок.	3.

У первой записи итерация всего одна, у второй их две.

Во втором разборе видно второе правило записи в работе: `null` занял одну ячейку и в очередь не попал, поэтому на второй итерации снимается двойка, а не он.

Про запись, которую разобрать нельзя.

Случай `[1, null, null, 5]` разобран выше, в разделе «Где это в коде проекта». В коде за него отвечает одна строка — `nodesToVisit.remove()`: очередь опустела раньше, чем кончилась запись, выдавать пятёрку некому, и метод останавливается с `NoSuchElementException`.

toLevelOrder : из дерева в запись

```
public static List<Integer> toLevelOrder(TreeNode root) {
    List<Integer> values = new ArrayList<>();
    if (root == null) {
        return values;
    }

    values.add(root.val);
    Queue<TreeNode> nodesToVisit = new ArrayDeque<>();
    nodesToVisit.add(root);

    /* У каждого непустого узла записываются оба потомка — значение или
    null. */
    while (!nodesToVisit.isEmpty()) {
        TreeNode parent = nodesToVisit.remove();
        for (TreeNode child : new TreeNode[]{parent.left, parent.right}) {
            if (child == null) {
                values.add(null);
            } else {
                values.add(child.val);
                nodesToVisit.add(child);
            }
        }
    }

    int lastValueIndex = values.size() - 1;
    while (values.get(lastValueIndex) == null) {
        lastValueIndex--;
    }
    return values.subList(0, lastValueIndex + 1);
}
```

Что происходит в коде, строка за строкой

Метод состоит из двух частей, и это видно по циклам: первый цикл записывает дерево целиком, второй отрезает хвост из `null`.

`List<Integer> values = new ArrayList<>()` — список под запись. Тип элемента `Integer`, а не `int`, потому что в записи есть `null`.

`if (root == null) return values` — у пустого дерева пустая запись. Это не охранный проверка: пустое дерево — законный вход, и пустой список — его законный ответ.

`values.add(root.val)` — значение корня записывается до цикла. Цикл записывает только потомков, а корень ничьим потомком не является — та же асимметрия, что и в `fromLevelOrder`.

`nodesToVisit.add(root)` — корень встаёт в очередь: его потомков ещё не записали.

`while (!nodesToVisit.isEmpty())` — здесь цикл идёт по очереди, а не по записи: работа кончается, когда потомки записаны у всех узлов. В `fromLevelOrder` было наоборот — там первой кончалась запись.

`TreeNode parent = nodesToVisit.remove()` — снят с головы узел, чьих потомков сейчас запишут.

`for (TreeNode child : new TreeNode[]{parent.left, parent.right})` — цикл по двум полям-ссылкам снятого узла, сначала левое, потом правое. Массив из двух ссылок создаётся только затем, чтобы не писать одно и то же тело дважды; он живёт одну итерацию внешнего цикла. Порядок полей в этом массиве и задаёт порядок «сначала левый, потом правый» в записи.

`if (child == null) { values.add(null); }` — пустой потомок записывается как `null` и в очередь не идёт: своих потомков у него быть не может.

`else { values.add(child.val); nodesToVisit.add(child); }` — непустой потомок записывается значением и встаёт в конец очереди, потому что его собственных потомков ещё предстоит записать.

`int lastValueIndex = values.size() - 1` — индекс последней ячейки списка, с которой начинается отрезание хвоста.

`while (values.get(lastValueIndex) == null) { lastValueIndex--; }` — индекс сдвигается влево, пока в ячейке лежит `null`. Останавливается он на последней ячейке со значением.

`return values.subList(0, lastValueIndex + 1)` — у `subList` нижняя граница включена, а верхняя нет, поэтому, чтобы последняя оставленная ячейка вошла в результат, верхняя граница берётся на единицу больше её индекса. Это та же единица, что в длине отрезка с включёнными границами, — см. `docs/index-arithmetic.md`.

Что делает каждая переменная.

Переменная.	Что хранит.	Зачем она нужна.
<code>root</code> .	корень дерева, может быть <code>null</code> .	вход метода; имя из условия LeetCode.
<code>values</code> .	накапливаемая запись, вместе с <code>null</code> .	ответ метода: корень записывается до цикла, потомки — в цикле.
<code>nodesToVisit</code> .	очередь узлов, чьих потомков ещё не записали.	та же роль, что в <code>fromLevelOrder</code> , с точностью до глагола: там потомков выдавали, здесь записывают.
<code>parent</code> .	узел, снятый с головы очереди.	у него читаются оба поля-ссылки
<code>child</code> .	очередной потомок снятого узла: сначала левый, потом правый.	по нему решается, что записать — значение или <code>null</code> .
<code>lastValueIndex</code> .	индекс последней ячейки, которая останется в записи.	сдвигается влево, пока упирается в <code>null</code> ; по нему считается верхняя граница <code>subList</code> .

Почему записываются оба потомка каждого непустого узла.

Потому что место в записи значимо. Разбирающий её `fromLevelOrder` на значения не смотрит: он читает ячейки по две на узел и раздаёт их узлам в порядке очереди. Значит, каждый `null` держит место, и убрать его нельзя — всё, что идёт дальше, сдвинется на ячейку влево и достанется не тому узлу.

Видно это на дереве `[1, null, 2]`:

```
1.
 \
 2.
```

Пропустить `null` — получится `[1, 2]`, а это уже другое дерево: двойка встанет левым потомком единицы. Одна пропущенная ячейка меняет форму дерева.

Отсюда и симметрия двух методов: `toLevelOrder` записывает по две ячейки на каждый непустой узел, `fromLevelOrder` по две и читает. Расходятся они только на хвосте: отрезанные `null` читать не из чего, поэтому узлам в конце очереди ячеек не достаётся — но эти `null` и значили «потомков нет», так что дерево выходит то же. Круг «из записи в дерево и обратно в запись» возвращает исходную запись; он проверен тестом на четырёх записях в `main()` класса `TreeNode`.

Почему хвостовые `null` отрезаются.

Из правила «оба потомка каждого непустого узла» следует, что до отрезания запись всегда кончается хвостом из `null`: у листьев последнего уровня потомков нет, а записать по два `null` на каждого метод обязан.

Отрезать этот хвост можно, и форма дерева от этого не меняется: после последней ячейки со значением не осталось ничего, что пропуск мог бы сдвинуть влево. Внутренний `null` держит место, хвостовой не держит ничего.

Отрезать его нужно по двум причинам:

- этого требует формат записи из условий LeetCode: `[1, 2]` и `[1, 2, null]` — одно и то же дерево, и в примерах пишут короткий вариант;
- без отрезания круг «из записи в дерево и обратно» не возвращал бы исходную запись: на входе была бы короткая запись, на выходе длинная.

Второй цикл не уходит за начало списка, и проверки `lastValueIndex >= 0` для этого не нужно. В ячейке с индексом 0 лежит значение корня, а корень не `null` — пустое дерево отсеяно проверкой в начале метода. Самое дальнее, куда цикл может пойти, — нулевая ячейка, и на ней он остановится. Так и происходит на дереве из одного узла: запись `[7, null, null]` отрезается до `[7]`, `lastValueIndex` останавливается на нуле. Это отдельный тест в `main()`.

Разбор по шагам.

То же дерево `[3, 9, 20, null, null, 15, 7]`:

Шаг.	Снятый узел.	Что записано.	Запись values в конце шага.	Очередь nodesToVisit в конце шага.
старт.	— нет.	значение корня.	3.	3.
итерация 1.	3.	левый потомок 9 и правый потомок 20, оба встали в очередь.	3, 9, 20.	9, 20.
итерация 2.	9.	потомков нет, записаны два null.	3, 9, 20, null, null.	20.
итерация 3.	20.	левый потомок 15 и правый потомок 7, оба встали в очередь.	3, 9, 20, null, null, 15, 7.	15, 7.
итерация 4.	15.	потомков нет, записаны два null.	3, 9, 20, null, null, 15, 7, null, null.	7.
итерация 5.	7.	потомков нет, записаны два null.	3, 9, 20, null, null, 15, 7, null, null, null, null.	пусто.

Очередь опустела, первый цикл закончился. В списке одиннадцать ячеек, и начинается отрезание хвоста:

Шаг	Индекс ячейки	Что лежит в ячейке	Что делает цикл
1.	10.	null.	сдвигает индекс влево.
2.	9.	null.	сдвигает индекс влево.
3.	8.	null.	сдвигает индекс влево.
4.	7.	null.	сдвигает индекс влево.
5.	6.	7.	не null, цикл останавливается.

`lastValueIndex` остался равен шести, и метод возвращает `subList(0, 7)` — это `[3, 9, 20, null, null, 15, 7]`, исходная запись.

Итераций пять, и узлов в дереве тоже пять. Совпадение не случайное: каждый узел попадает в очередь ровно один раз и ровно один раз с неё снимается. У `fromLevelOrder` на том же дереве итераций было три — там цикл останавливает не очередь, а конец записи.

Из одиннадцати записанных ячеек в ответе осталось семь: четыре хвостовых `null` отрезаны, по два от листьев `15` и `7`.

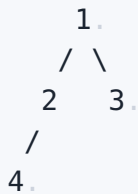
Чем очередь здесь отличается от очереди в задаче 102.

Очередь устроена одинаково: `ArrayDeque` в роли `Queue`, порядок FIFO, потомки дописываются в конец, пустые узлы в очередь не попадают. Различается то, сколько узлов снимается за одну итерацию цикла, и следует это прямо из формы ответа.

В `LevelOrderTraversal` ответ разбит по уровням, поэтому границы уровней нужно знать. Там перед внутренним циклом снимается размер очереди, и внутренний цикл берёт ровно столько узлов — целый уровень. Циклов два, вложенных один в другой.

Здесь ответ плоский: в записи по уровням границы между уровнями ничем не отмечены. Знать их не нужно, поэтому цикл один и за итерацию снимается один узел.

Отсюда следствие, которое проще увидеть на примере, чем вывести. В задаче 102 очередь в начале итерации внешнего цикла содержит ровно один уровень целиком. Здесь такого не бывает даже случайно: в разборе `toLevelOrder` выше в конце итерации 2 в очереди лежит один узел 20 — половина второго уровня. А на дереве [1, 2, 3, 4].



в конце второй итерации в очереди лежат 3 и 4 — узлы разных уровней: 3 со второго, 4 с третьего.

	<code>fromLevelOrder</code> и <code>toLevelOrder</code> .	<code>LevelOrderTraversal</code> , задача 102.
Снимается за итерацию.	один узел.	целый уровень.
Циклов по очереди.	один.	два, вложенных.
Отмечены ли границы уровней.	нет, запись плоская.	да, размер уровня снимается до внутреннего цикла.
Что лежит в очереди.	узлы, чьих потомков ещё не выдали или не записали.	узлы, чьи значения ещё не записаны в ответ.
Чем кончается работа.	<code>toLevelOrder</code> — очередь опустела; <code>fromLevelOrder</code> — кончилась запись, очередь при этом может быть не пуста.	очередь опустела.
Попадают ли <code>null</code> в очередь.	нет.	нет.
Попадают ли <code>null</code> в ответ.	да, <code>toLevelOrder</code> записывает их как ячейки-места.	нет, ответ состоит только из значений узлов.

Приём 10 из шпаргалки — «цикл меняет то, по чему идёт, зафиксируй границу до входа» — нужен только правой колонке. В `toLevelOrder` очередь тоже меняется внутри цикла, но её размер ни на что не влияет: условие `!nodesToVisit.isEmpty()` спрашивает не «сколько», а «есть ли хоть один». Фиксировать нечего.

Посмотреть в движении.

docs/visualizations/block06_trees_graphs/TreeNotation.html — пошаговый разбор записи: массив с подсветкой читаемой ячейки, список ожидающих и дерево, которое растёт на каждом шаге. Там же панель с кодом `fromLevelOrder` и подсветкой текущей строки.

См. также.

- [MaxDepthBinaryTree.md](#) — первая задача блока, где эта запись встречается.
- [LevelOrderTraversal.md](#) — задача 102: та же очередь, но снимается целый уровень за итерацию.
- [Pattern_GraphTraversal.md](#) — обзор блока: обходы в глубину и в ширину, какой когда.
- [docs/datastructures/Queue.md](#) — очередь, на которой держится и разбор записи, и сам обход в ширину.